

Taller de Git!

ComCom

DC - FCEyN - UBA

September 18, 2026



taller
de
git

Trabajando en grupos

IMPORTANTE

Hoy vamos a hacer varios ejercicios donde trabajamos de a grupos. Es importante que se sienten cerca de alguien con quien van a hacer unos ejercicios hoy. Vamos a configurar cosas de la compu y va a ser difícil mudarse mas tarde.

Configuraciones iniciales 1/3

Tu identidad

Es importante establecer nuestro **nombre y email** en nuestro repositorio, ya que estos van a ir asociados con los cambios que hagamos:

```
git config --global user.name "Guybrush Threepwood"  
git config --global user.email guybrush@example.com
```

Modo de pull

Ejecuten esto (es una mauskiherramienta que nos ayudara mas tarde)

```
git config --global pull.rebase false
```

Config global vs local

También se puede omitir el flag `--global` para que las credenciales apliquen solo al repositorio en el que estamos trabajando.

Configuraciones iniciales 2/3

Clave SSH

Para poder trabajar cómodamente con repositorios Git que estén en Internet (GitHub, Bitbucket, GitLab, etc.), podemos configurar una **clave SSH** que nos identifique con el servidor que estemos usando. De esta manera, puede reconocer desde que usuario vienen los pedidos (para revisar permisos, etc...).

Configuraciones iniciales 3/3

Creando una clave nueva

Abrimos la terminal en el HOME y ejecutamos `cd .ssh` y luego `ssh-keygen -f id_rsa`. Le damos *Enter* a todo hasta que termine el proceso.

Configuraciones iniciales 3/3

Creando una clave nueva

Abrimos la terminal en el HOME y ejecutamos `cd .ssh` y luego `ssh-keygen -f id_rsa`. Le damos *Enter* a todo hasta que termine el proceso.

Subiendo la clave a COMCOMHub

- En la terminal, ejecutamos `cat id_rsa.pub` y copiamos todo lo que aparezca.
- Abrimos COMCOMHub, vamos a Configuración, Claves SSH / GPG, Añadir clave.
- Pegamos lo que tenemos copiado en el campo *Contenido*.

Configuraciones iniciales 3/3

Creando una clave nueva

Abrimos la terminal en el HOME y ejecutamos `cd .ssh` y luego `ssh-keygen -f id_rsa`. Le damos *Enter* a todo hasta que termine el proceso.

Subiendo la clave a COMCOMHub

- En la terminal, ejecutamos `cat id_rsa.pub` y copiamos todo lo que aparezca.
- Abrimos COMCOMHub, vamos a Configuración, Claves SSH / GPG, Añadir clave.
- Pegamos lo que tenemos copiado en el campo *Contenido*.

Funcionó?

Pueden probar que la clave quedó funcionando usando el siguiente comando: `ssh -p 222 git@ssh.cubawiki.com.ar`. Deberían ver su username de COMCOMHub.

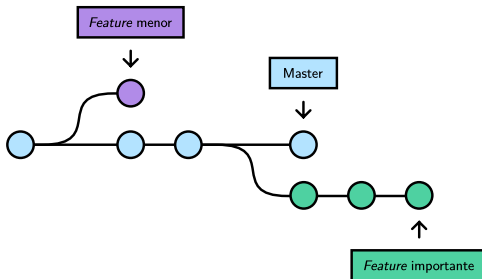
Ramificaciones en Git

Una **rama** (o *branch*) en Git representa una línea independiente de desarrollo. Al crear nuevas ramas, podemos pensar que nuestro proyecto diverge en dos distintos: los cambios que hagamos en uno no impactan al otro.

Ramificaciones en Git

Una **rama** (o *branch*) en Git representa una línea independiente de desarrollo. Al crear nuevas ramas, podemos pensar que nuestro proyecto diverge en dos distintos: los cambios que hagamos en uno no impactan al otro.

Un ejemplo visual:



Creando ramas

git branch

Creando ramas

git branch

Para crear una rama nueva, podemos ejecutar `git branch [nombre de la rama]`.

Nótese que este comando no nos mueve a la nueva rama, solo la crea.

Para ver las ramas de nuestro repositorio local, podemos ejecutar `git branch`.

Creando ramas

git branch

Para crear una rama nueva, podemos ejecutar `git branch [nombre de la rama]`.

Nótese que este comando no nos mueve a la nueva rama, solo la crea.

Para ver las ramas de nuestro repositorio local, podemos ejecutar `git branch`.

Ejercicio

Crear una rama llamada “prueba” en algún repo de la clase pasada.

Cambiando de rama

`git checkout`

Cambiando de rama

git checkout

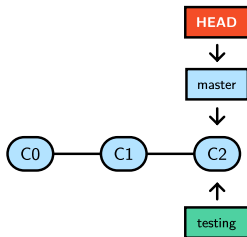
Para cambiar de rama, podemos ejecutar `git checkout [nombre de la rama]`.

Cambiando de rama

git checkout

Para cambiar de rama, podemos ejecutar `git checkout [nombre de la rama]`.

Un ejemplo visual:

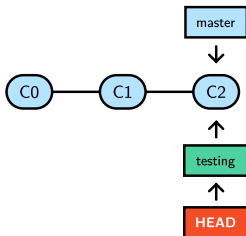


Cambiando de rama

git checkout

Para cambiar de rama, podemos ejecutar `git checkout [nombre de la rama]`.

Un ejemplo visual:



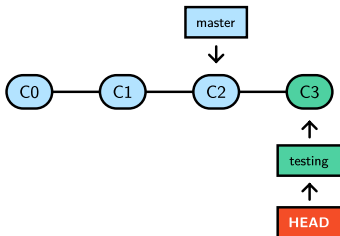
Acá cambiamos a la rama “testing”, ejecutando `git checkout testing`.

Cambiando de rama

git checkout

Para cambiar de rama, podemos ejecutar `git checkout [nombre de la rama]`.

Un ejemplo visual:



Y los siguientes *commits* serán agregados a la rama "testing".

Fusionando ramas

`git merge`

Fusionando ramas

git merge

Nos permite fusionar las historias de dos ramas distintas (podría haber conflictos).

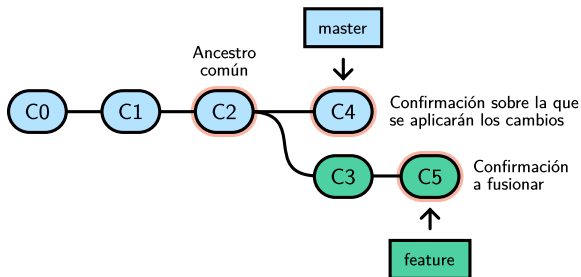
La sintaxis es: `git merge [nombre de la rama a fusionar]`.

Importante: este comando fusiona la rama que le decimos **en la rama en la que estamos parados**.

Fusionando ramas

git merge

Un ejemplo visual:

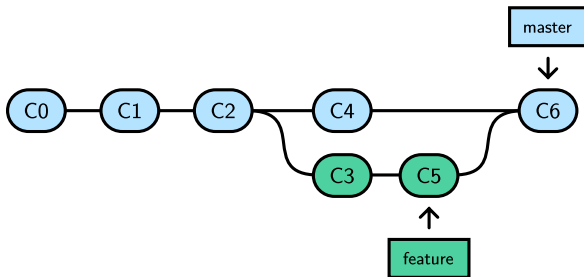


Antes del *merge*.

Fusionando ramas

git merge

Un ejemplo visual:



Después de pararnos en la rama “master” (`git checkout master`) y haber fusionado la rama “feature” (`git merge feature`).

¡A practicar!

¡A practicar!

Pero antes de empezar...

Para hacer el siguiente ejercicio, vamos a trabajar con un **fork** de un repositorio.

¡A practicar!

Pero antes de empezar...

Para hacer el siguiente ejercicio, vamos a trabajar con un **fork** de un repositorio. ¿Y eso?

¡A practicar!

Pero antes de empezar...

Para hacer el siguiente ejercicio, vamos a trabajar con un **fork** de un repositorio. ¿Y eso?

Los servidores de Git, como GitLab, nos permiten hacer *fork* de proyectos. Al hacer esto creamos una copia de un repositorio ajeno en nuestra propia cuenta, sobre la cual tendremos todos los permisos para pushear libremente. Los cambios que hagamos en nuestra copia **no** impactarán en el repositorio original.

¡A practicar!

Pero antes de empezar...

Para hacer el siguiente ejercicio, vamos a trabajar con un **fork** de un repositorio. ¿Y eso?

Los servidores de Git, como GitLab, nos permiten hacer *fork* de proyectos. Al hacer esto creamos una copia de un repositorio ajeno en nuestra propia cuenta, sobre la cual tendremos todos los permisos para pushear libremente. Los cambios que hagamos en nuestra copia **no** impactarán en el repositorio original.

Ahora sí...

¡A practicar!

Ejercicio de a 2 máquinas (preferiblemente 2 personas): 🦹 y 🦿

¡A practicar!

Ejercicio de a 2 máquinas (preferiblemente 2 personas): 🧐 y 🦾

- 1 🧐: Hacer un *fork* en su cuenta de COMCOMHub de este repositorio: <https://git.cubawiki.com.ar/talleres-comcom/taller-git-ejercicio2>, y darle permiso a 🦾 para hacer *push* (por lo menos *Escritura*).

¡A practicar!

Ejercicio de a 2 máquinas (preferiblemente 2 personas): 🧐 y 🐙

- 1 🧐: Hacer un *fork* en su cuenta de COMCOMHub de este repositorio: <https://git.cubawiki.com.ar/talleres-comcom/taller-git-ejercicio2>, y darle permiso a 🐙 para hacer *push* (por lo menos *Escritura*).
- 2 🧐 y 🐙: Obtener una copia local del repositorio de 🧐 (*git clone ...* y *cd taller-...*)

¡A practicar!

Ejercicio de a 2 máquinas (preferiblemente 2 personas): 🧐 y 🦾

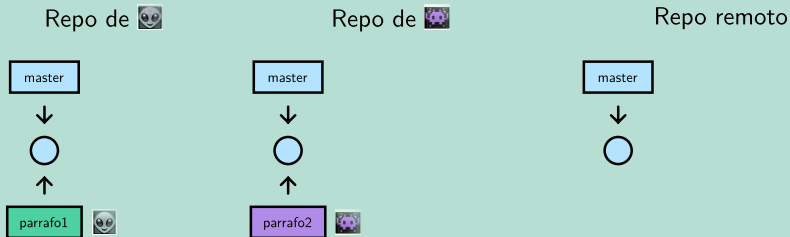
- 1 🧐: Hacer un *fork* en su cuenta de COMCOMHub de este repositorio: <https://git.cubawiki.com.ar/talleres-comcom/taller-git-ejercicio2>, y darle permiso a 🦾 para hacer *push* (por lo menos *Escritura*).
- 2 🧐 y 🦾: Obtener una copia local del repositorio de 🧐 (*git clone ...* y *cd taller-...*)
- 3 🧐 y 🦾: El repo tiene una única rama, “master”, donde van a encontrar un fragmento incompleto, de un cuento de Borges. Repartirse el trabajo: cada uno deberá completar un párrafo.



¡A practicar!

Ejercicio de a 2 máquinas (preferiblemente 2 personas): 🧐 y 🐙

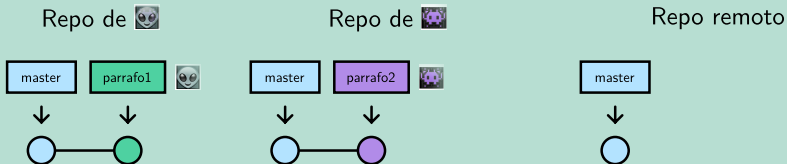
- 1 🧐: Hacer un *fork* en su cuenta de COMCOMHub de este repositorio: <https://git.cubawiki.com.ar/talleres-comcom/taller-git-ejercicio2>, y darle permiso a 🐙 para hacer *push* (por lo menos *Escritura*).
- 2 🧐 y 🐙: Obtener una copia local del repositorio de 🧐 (*git clone ...* y *cd taller-...*)
- 4 🧐 y 🐙: Crear, cada uno, una rama propia donde harán sus modificaciones (con *branch*). Posicionarse en la rama recién creada (con *checkout*).



¡A practicar!

Ejercicio de a 2 máquinas (preferiblemente 2 personas): 🧑 y 🦾

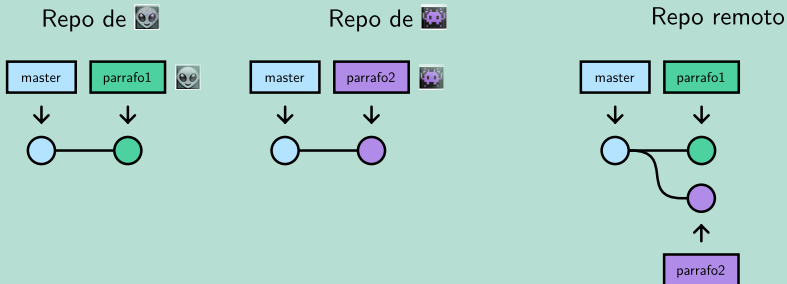
- 1 🧑: Hacer un *fork* en su cuenta de COMCOMHub de este repositorio:
<https://git.cubawiki.com.ar/talleres-comcom/taller-git-ejercicio2>, y darle permiso a 🦾 para hacer *push* (por lo menos *Escritura*).
- 2 🧑 y 🦾: Obtener una copia local del repositorio de 🧑 (*git clone ...* y *cd taller-...*)
- 5 🧑 y 🦾: Completar la parte elegida del cuento, y hacer *commit* y *push* de estos cambios en el repositorio remoto.



¡A practicar!

Ejercicio de a 2 máquinas (preferiblemente 2 personas): 🧐 y 🦾

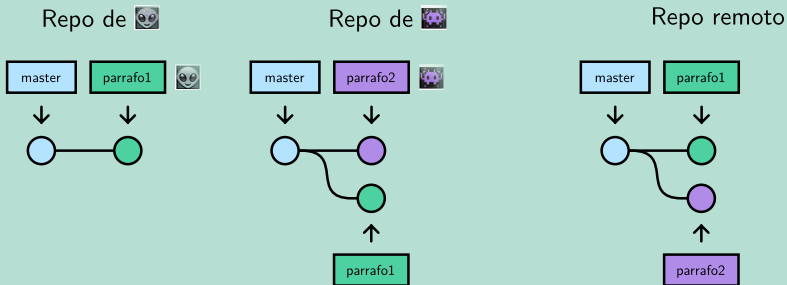
- 1 🧐: Hacer un *fork* en su cuenta de COMCOMHub de este repositorio:
<https://git.cubawiki.com.ar/talleres-comcom/taller-git-ejercicio2>, y darle permiso a 🦾 para hacer *push* (por lo menos *Escritura*).
- 2 🧐 y 🦾: Obtener una copia local del repositorio de 🧐 (*git clone ...* y *cd taller-...*)
- 5 🧐 y 🦾: Completar la parte elegida del cuento, y hacer *commit* y *push* de estos cambios en el repositorio remoto.



¡A practicar!

Ejercicio de a 2 máquinas (preferiblemente 2 personas): 🧑🏻 y 🦾

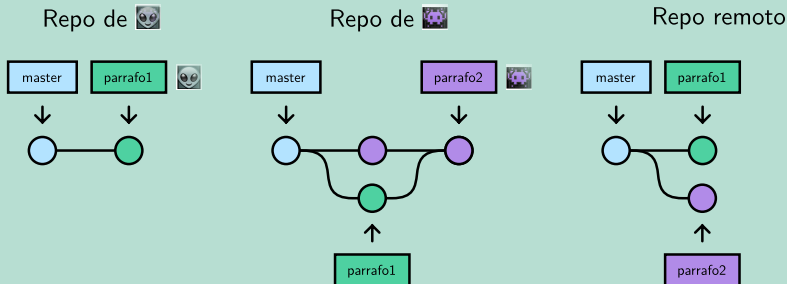
- 1 🧑🏻: Hacer un *fork* en su cuenta de COMCOMHub de este repositorio: <https://git.cubawiki.com.ar/talleres-comcom/taller-git-ejercicio2>, y darle permiso a 🦾 para hacer *push* (por lo menos *Escritura*).
- 2 🧑🏻 y 🦾: Obtener una copia local del repositorio de 🧑🏻 (*git clone ...* y *cd taller-...*)
- 6 🦾: Traer los cambios de la rama de 🧑🏻.



¡A practicar!

Ejercicio de a 2 máquinas (preferiblemente 2 personas): 🧐 y 🦾

- 1 🧐: Hacer un *fork* en su cuenta de COMCOMHub de este repositorio: <https://git.cubawiki.com.ar/talleres-comcom/taller-git-ejercicio2>, y darle permiso a 🦾 para hacer *push* (por lo menos *Escritura*).
- 2 🧐 y 🦾: Obtener una copia local del repositorio de 🧐 (*git clone ...* y *cd taller-...*)
- 7 🦾: Fusionar en la rama de 🦾 los cambios de 🧐. Enviar estos cambios al repositorio remoto.



¡A practicar!

Ejercicio de a 2 máquinas (preferiblemente 2 personas): 🧐 y 🐙

- 1 🧐: Hacer un *fork* en su cuenta de COMCOMHub de este repositorio: <https://git.cubawiki.com.ar/talleres-comcom/taller-git-ejercicio2>, y darle permiso a 🐙 para hacer *push* (por lo menos *Escritura*).
- 2 🧐 y 🐙: Obtener una copia local del repositorio de 🧐 (*git clone ...* y *cd taller...*)
- 8 🧐: Seguir los dos pasos anteriores, pero con los cambios de 🐙.

¡A practicar!

Ejercicio de a 2 máquinas (preferiblemente 2 personas): 🧐 y 🐙

- 1 🧐: Hacer un *fork* en su cuenta de COMCOMHub de este repositorio: <https://git.cubawiki.com.ar/talleres-comcom/taller-git-ejercicio2>, y darle permiso a 🐙 para hacer *push* (por lo menos *Escritura*).
- 2 🧐 y 🐙: Obtener una copia local del repositorio de 🧐 (*git clone ...* y *cd taller-...*)
- 3 🧐 y 🐙: El repo tiene una única rama, “master”, donde van a encontrar un fragmento incompleto, de un cuento de Borges. Repartirse el trabajo: cada uno deberá completar un párrafo.
- 4 🧐 y 🐙: Crear, cada uno, una rama propia donde harán sus modificaciones (con *branch*). Posicionarse en la rama recién creada (con *checkout*).
- 5 🧐 y 🐙: Completar la parte elegida del cuento, y hacer *commit* y *push* de estos cambios en el repositorio remoto.
- 6 🐙: Traer los cambios de la rama de 🧐.
- 7 🐙: Fusionar en la rama de 🐙 los cambios de 🧐. Enviar estos cambios al repositorio remoto.
- 8 🧐: Seguir los dos pasos anteriores, pero con los cambios de 🐙.

Revirtiendo cambios 1/2

git commit --amend

Podemos usarlo para **arreglar**, por ejemplo, el mensaje del último *commit* que hicimos: `git commit --amend -m [nuevo mensaje]`. También sirve para añadir archivos al último commit que nos hayamos olvidado.

Revirtiendo cambios 1/2

git commit --amend

Podemos usarlo para **arreglar**, por ejemplo, el mensaje del último *commit* que hicimos: `git commit --amend -m [nuevo mensaje]`. También sirve para añadir archivos al último commit que nos hayamos olvidado.

Ejemplo:

```
git commit -m 'Commit inicial'  
git add archivo-olvidado  
git commit --amend
```

Revirtiendo cambios 2/2

git revert

Permite **revertir** exactamente los cambios introducidos por un *commit*. Buscamos el *hash* del *commit* en cuestión usando `git log`, y luego ejecutamos `git revert [hash]`. También podemos deshacer uno o varios commits relativos al HEAD.

Ejemplos:

```
git revert HEAD~3
```

Revierte los cambios del 4to commit anterior al HEAD. Crea un commit nuevo con los cambios.

Revirtiendo cambios 2/2

git revert

Permite **revertir** exactamente los cambios introducidos por un *commit*. Buscamos el *hash* del *commit* en cuestión usando `git log`, y luego ejecutamos `git revert [hash]`. También podemos deshacer uno o varios commits relativos al HEAD.

Ejemplos:

```
git revert HEAD~3
```

Revierte los cambios del 4to commit anterior al HEAD. Crea un commit nuevo con los cambios.

```
git revert -n master~5..master~2
```

Revierte los cambios del 5to al 3er commit en la rama master (incluido). La flag `-n` hace que no cree un commit nuevo con los cambios.

Otro comando útil

```
git stash
```

Otro comando útil

git stash

Al ejecutar `git stash`, se guarda el estado actual de los archivos modificados y nos deja el directorio limpio.

Para volver a mostrar los cambios guardados ejecutamos `git stash apply`.

Extras: Ignorando archivos

El archivo .gitignore

Es común tener archivos generados automáticamente que no queremos agregar al repositorio. Por ejemplo, archivos compilados: .pdf, .exe, .log, .pyc, etc. Sin embargo, es bastante molesto verlos todo el tiempo al ejecutar `git status`.

Para arreglar esto podemos crear un archivo especial llamado `.gitignore`, que le indica a Git qué archivos **ignorar** por completo.

Extras: Ignorando archivos

El archivo .gitignore

Es común tener archivos generados automáticamente que no queremos agregar al repositorio. Por ejemplo, archivos compilados: `.pdf`, `.exe`, `.log`, `.pyc`, etc. Sin embargo, es bastante molesto verlos todo el tiempo al ejecutar `git status`.

Para arreglar esto podemos crear un archivo especial llamado `.gitignore`, que le indica a Git qué archivos **ignorar** por completo.

Por ejemplo, para ignorar todos los archivos con extensión `.pyc`:

- 1 Crear un archivo llamado `.gitignore` en el directorio principal del proyecto.
- 2 Adentro escribir: `*.pyc`

Más ejemplos de `.gitignore`:

<https://github.com/github/gitignore>.

Prueben los comandos que vimos recién

Ejercicio 1

- 1  y : En el mismo repo de antes, sitúense en ramas diferentes.
- 2 : Creá un archivo llamado *perrito*, e incluílo en un commit nuevo, con un mensaje poco legible.
- 3 : Creá un archivo llamado *gatito*, e incluílo en un commit nuevo, con un mensaje poco legible.
- 4  y : Pushear cambios y traerse los cambios del otro.
- 5  y : Cambienle el texto al commit de la otra persona con uno que se entienda más. Recuerden el comando `git commit --amend -m [nuevo mensaje]`.
- 6  y : Hagan push (si tira un mensaje de error prueben hacer `git push -f`), y vean como cambió su historial de commits tanto en la web como local.

Extras: Más comandos

- `git fetch [remote repository]`: Para traer todos los datos de un repositorio remoto.
- `git reset`: Permite deshacer cambios; sirve para revertir modificaciones en el área de trabajo, pero también puede eliminar por completo *commits* anteriores. ¡Usar con mucho cuidado!
- `git rebase [branch]`: Aplica todos los *commits* que difieren entre una rama y aquella en la que estamos parados. Así podemos incorporar cambios realizados en otras ramas manteniendo lineal el historial de la rama actual.
- `git blame [archivo]`: Para ver qué *commit* modificó por última vez cada línea de un archivo, y quién fue su autor. ¡Así, podemos saber a quién *culpar* cuando haya problemas!
- `git bisect`: Encuentra cuál fue el *commit* que introdujo cierto error, haciendo búsqueda binaria en el historial de *commits*.

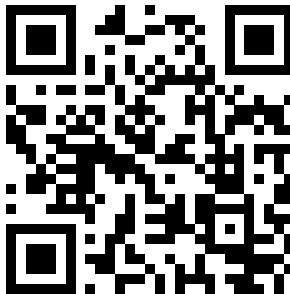
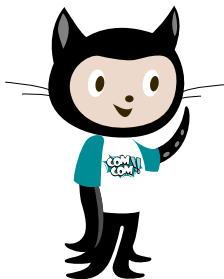
Extras: Servidores



Bibliografía

- Git Community book, disponible online y en español:
<https://git-scm.com/book/es/v2>
- `git help [command]` para ver la documentación de cualquier comando de Git.
- A visual Git reference: <http://marklodato.github.io/visual-git-guide/index-es.html>

Feedback



¡Nos interesa saber que les pareció el taller!

Pueden completar el formulario:

<https://forms.gle/6BoJUyyUDBMi5Edp8>

También nos pueden tirar ideas para futuros talleres.